Softwerk AB – Coding Test

AI Engineer

Introduction

Our lead detective has a problem: He has collected thousands of case files, witness statements, and clues over the years, all stored as digital text documents. But he's forgetting the details. He needs a way to instantly ask questions about his old cases without re-reading thousands of pages.

Your mission: Build **"Sherlock,"** a digital assistant that can read through the detective's case files and answer questions about suspects, alibis, and clues.

The Scenario

The Detective wants to upload a case file (a PDF or text document containing witness interviews or police reports) and immediately ask:

- *"What was Mrs. Hudson's alibi?"*
- *"Who was seen leaving the manor at midnight?"*
- *"Did the butler have a motive?"*

The Catch: The Detective hates guesses. If the answer isn't in the case file, the assistant must say, **"I don't have enough evidence to answer that."** It cannot hallucinate or make up clues.

The Requirements

You are building a technical solution for The Detective, with the following requirements:

1. **The Interface:** The system must expose an API (REST or similar) that allows a user to:
    - Add new documents (Case Files) to the knowledge base.
    - Ask questions derived from those documents.

2. **The Intelligence:** The system must use an LLM to generate natural language answers. However, it must use **Retrieval Augmented Generation (RAG)** (or a similar technique) to ensure answers are strictly based on the provided documents.
3. **The Infrastructure:**
    - Language: **Python**.
    - Delivery: A **Docker container**. You must provide the `Dockerfile` and the commands needed to build and run it.
4. **The front-end**: The system should have a small front-end component that is connected to the API interface and spins up as part of the Docker container. Any tech stack is allowed here, but modern tech stacks are recommended.

Scope & Constraints

- **Time:** We estimate this task to take **8–12 hours**.
- **Language:** Python 3.10+.
- **Delivery:** A GitHub repository containing the code, and clear instructions on how to run it. Make sure to commit frequently.
- **Cost:** You should not need to spend money. Use the free tiers of API providers (e.g., Google AI Studio, HuggingFace, etc.).
- **AI Usage:** We love AI, but AI usage for this coding project should be limited to minimal usage . We want to understand your thinking, decision making and coding.

How to deliver

- Upload your work to GitHub and provide us with a link.
- The repository should contain clear instructions on how to run the project, with additional keys required.
- A short final report on your project can be added to the email you send together with your GitHub repository.

Scoring is done from the following aspects

- **Solved:** Was the problem "solved" i.e. does it work according to the instructions?
- **Code:** Is the code nice, tidy and easy to follow, using good coding practices?
- **Architecture:** Is the architecture reasonable?
- **Modern technologies:** Are modern technologies being used?
- **Framework leverage:** Were suitable frameworks leveraged?
- **Documentation:** Is the code and how to use the program sufficiently documented?
- **Test:** Is the code sufficiently tested?
- **Bonus:** Any nice bonus features?